Maw3edak

Healthcare Appointment & Clinic Management — Mobile App

Full Technical Documentation

Version 1.0.0 | React Native · Expo | Supabase + Firebase | June 2026

This document describes the complete Maw3edak mobile application — its architecture, screens, design system, frontend code, backend serverless functions, and database. Use the **Save as PDF** button (top-right) or your browser's **Print** → **Save as PDF** to export this as a PDF / document.

50+	110+	12	130+	2
APP SCREENS	EDGE FUNCTIONS	CUSTOM TABLES	DB MIGRATIONS	USER ROLES

Contents

1. Project Overview

Maw3edak (Arabic for “your appointment”) is a cross-platform healthcare application that connects **patients** with **doctors / clinics**. It supports the full clinical journey: discovering doctors, booking appointments, accessing medical records (prescriptions, dental, vision, nutrition), managing invoices and payments, answering medical questionnaires, and receiving real-time push notifications.

The app ships **two complete experiences** from a single codebase, selected at login:

Patient App

Book & manage appointments, browse doctors, view prescriptions / invoices / packages / statements, dental & vision & nutrition records, doctor Q&A responses, profile & payment methods, notifications.

Doctor / Clinic App

Dashboard with daily stats, today's & all appointments, patient list, prescriptions, orders, finance summary, time-management (schedules), questionnaires & patient answers, authorization workflow, settings.

CORE FEATURES

Patient & Doctor authentication · OTP phone/email verification · Biometric login gate · Multi-device login · Multi-clinic switching · Appointment booking engine · FCM push notifications · Medical records access · Prescriptions & orders · Invoices & statements · Questionnaires · Dark mode · Card scanner (camera)

2. Technology Stack

Layer	Technology	Purpose
Language	TypeScript / React 19	Type-safe app code
Framework	React Native 0.81 + Expo SDK 54	Cross-platform iOS / Android / Web
Navigation	Expo Router 6 (file-based, typed routes)	Stack + bottom tabs routing
Backend	Supabase (PostgreSQL + Edge Functions/Deno)	Database, auth sessions, serverless API
Push	Firebase Cloud Messaging + expo-notifications	Real-time notifications
Auth storage	AsyncStorage + expo-secure-store	Session persistence & secrets
Biometrics	expo-local-authentication	Face ID / fingerprint gate
UI / Icons	lucide-react-native, @expo/vector-icons, expo-linear-gradient, expo-blur	Visual layer
Fonts	Inter (Regular / Medium / SemiBold / Bold)	Typography
Camera / Images	expo-camera, expo-image-picker	Card scan, profile photos
Crypto	expo-crypto, @noble/hashes	Password hashing
Build / OTA	EAS Build, expo-updates	Native builds & over-the-air updates

3. Architecture

The system follows a classic three-tier model. The mobile client never talks to the database directly — every privileged operation goes through a **Supabase Edge Function** that validates a custom session token, then uses the service role to read/write Postgres.

REQUEST FLOW (EXAMPLE: BOOK APPOINTMENT)

- Patient taps **Book** → screen calls `mobile-book-appointment` with the session token in the `X-Session-Token` header.
- The edge function validates the token against `user_patient_sessions`, inserts the appointment, and returns a result.
- A Postgres **trigger** fires `notify_patient_new_appointment`, which inserts a `patient_notifications` row.
- An **AFTER INSERT** trigger (`send_fcm_on_patient_notification`) uses `pg_net` to call `send-push-notification`, delivering the FCM message to the device.

---

4. Design System

All visual tokens are centralized in `constants/theme.ts` and exposed at runtime through `ThemeContext` (which also provides light/dark variants).

Color Palette

Primary	Primary Dark	Primary Light	Success	Warning
#006BFF	#1E40AF	#EFF6FF	#10B981	#F59E0B

Error	Purple	Text	Text Sec.	BG Secondary
#EF4444	#8B5CF6	#1E293B	#64748B	#F8FAFC

CATEGORY COLORS (NOTIFICATION & RECORD TYPES)

`order` `appointment` `prescription` `package` `invoice` `question` `general`

Typography (Inter)

Token	Size	Weight	Line height
h1	28	Bold	36
h2	24	Bold	32
h3	20	Bold	28
h4	18	SemiBold	24
body	15	Regular / Medium / SemiBold	22
caption	13	Regular / Medium	18
small	11	Regular / Medium	16

Spacing & Radius scale

Spacing: `xs 4` `sm 8` `md 12` `lg 16` `xl 20` `xxl 24` `xxxl 32`

Border radius: `sm 8` `md 12` `lg 16` `xl 20` `xxl 24` `full 9999`

Elevation/shadow: three presets (`sm` , `md` , `lg`) with platform-aware iOS shadow + Android elevation.

---

5. Navigation & App Structure

Routing is file-based (Expo Router). The root `app/_layout.tsx` wraps the app in `ErrorBoundary → SafeAreaProvider → AuthProvider → ThemeProvider → BiometricAuthGate` , initializes Firebase & notification listeners, loads Inter fonts, and manages the splash screen.

```
app/
├── _layout.tsx              Root providers, Firebase, fonts, splash
├── index.tsx                Entry / session routing
├── login.tsx                Role selection entry
├── (auth)/                  ── Authentication flow ──
│   ├── patient-login.tsx        doctor-login.tsx
│   ├── patient-register.tsx     registration-success.tsx
│   ├── otp-verify.tsx           forgot-password.tsx
│   ├── reset-password.tsx       reset-password-otp.tsx
├── (tabs)/                  ── PATIENT experience (bottom tabs) ──
│   ├── index (Home) · appointments · visit-history · settings
│   └── [stacked] profile, doctors, prescriptions, orders, invoices,
│       statement, packages, dental, vision, nutrition, doctor-responses
├── (doctor-tabs)/           ── DOCTOR experience ──
│   ├── index (Dashboard) · appointments · patients · settings
│   └── [stacked] questions, questionnaires, patient-answers, finance,
│       time-management, orders, medications, dental, vision, nutrition,
│       notifications, patient-appointments
└── [modals]  book-appointment · edit-profile · package-detail ·
              payment-methods · notifications · help-center
```

> **Visible tabs (patient):** Home, Appointments, Visits, (Settings/Profile accessed via header). Other routes use `href:null` so they're navigable but hidden from the tab bar. Android hardware back is intercepted to exit the app from root tabs and navigate normally elsewhere.

6. Screens Catalog

Authentication `app/(auth)`

Screen	Purpose
patient-login	Patient login (medical ID / phone + password)
doctor-login	Doctor / staff login
patient-register	New patient registration with image captcha
otp-verify	Verify phone/email OTP code
forgot-password / reset-password / reset-password-otp	Password recovery flow via OTP
registration-success	Confirmation screen after sign-up

Patient `app/(tabs)`

Screen	Purpose
index (Home)	Dashboard: upcoming appointments, quick actions, shortcuts
appointments	List/manage appointments, status & actions
visit-history	Past visits timeline
doctors	Browse / search accessible doctors
prescriptions	Patient prescriptions
orders	Medical/pharmacy orders
invoices / invoices detail	Billing documents
statement	Financial statement / ledger
packages	Treatment packages
dental / vision / nutrition	Specialty medical records
doctor-responses	Answers to patient questions
profile / settings	Account, preferences, dark mode, notifications, biometrics

Doctor / Clinic `app/(doctor-tabs)`

Screen	Purpose
index (Dashboard)	Today's schedule + stats (pending, completed, cancelled, total patients)
appointments / patient-appointments	All & per-patient appointments + actions
patients	Doctor's patient list with details
time-management	Recurring schedules, exceptions, blocks, slot duration
questions / questionnaires	Create & manage Q&A and questionnaires
patient-answers	Review submitted questionnaire answers
finance	Finance summary
orders / medications	Doctor orders & prescriptions
dental / vision / nutrition	Manage specialty encounters & plans
notifications	Doctor notifications & authorization workflow
settings	Security settings, notifications toggle

Shared modals / utility

book-appointment · edit-profile · package-detail · payment-methods · notifications · help-center · +not-found

7. Reusable Components

Component	Responsibility
BiometricAuthGate	Wraps the app; enforces Face ID / fingerprint unlock when enabled. Exposes `resetBiometricSession()`.
ErrorBoundary	Catches render errors and shows a safe fallback UI.
CardScannerModal	Camera-based payment card scanner (expo-camera).
ImageCaptcha	Image-based CAPTCHA for registration anti-bot.
FilterBottomSheet	Reusable bottom-sheet filter UI.
NotificationCard	List row for a single notification (category color, read state).
NotificationDetail	Expanded notification view + deep-link navigation.

8. State, Hooks & Utilities

Contexts

AuthContext

Holds `session` (custom) & `supabaseSession`; exposes `signOut`, `refreshSession`, `switchClinic`. Bootstraps from AsyncStorage with timeout guards, listens to Supabase auth changes, deactivates the device token on sign-out.

ThemeContext

Provides resolved `colors` for light/dark mode, persists the user's dark-mode preference, and syncs it to the server (`mobile-update-darkmode`).

Hooks

`useFrameworkReady()` — framework readiness signal used during boot.

Utilities `utils/`

File	Responsibility
supabase.ts	Creates the Supabase client (AsyncStorage persistence, auto-refresh); falls back to a safe dummy client on init failure.
config.ts	Centralized config (Supabase URL / anon key) from Expo `extra`.
auth.ts	Session model + `saveSession / getSession / clearSession`.
firebase.ts	Initializes Firebase app (native only).
notifications.ts	Notification listeners, background handler, navigation routing.
passwordHash.ts	Password hashing (noble/hashes + expo-crypto).
settingsService.ts	Read/write user settings (biometrics, notifications, dark mode).
validation.ts	Form validation helpers.
dateFormat.ts	Date/time formatting.

---

9. Backend — Supabase Edge Functions

110+ Deno/TypeScript serverless functions under `supabase/functions/`. Each validates the caller, performs DB work via the service role, and returns JSON `{ success, data | error }`. Grouped by domain below.

Authentication & Session

`mobile-login` `mobile-patient-login` `doctor-mobile-login` `patient-register` `mobile-send-otp` `mobile-verify-otp` `mobile-check-email` `mobile-check-phone` `mobile-change-password` `mobile-reset-password` `mobile-get-phone-by-medical-id` `mobile-handle-authorization` `mobile-logout-device` `mobile-switch-company`

Appointments & Booking

`mobile-book-appointment` `mobile-get-available-dates` `mobile-get-available-slots` `mobile-get-doctors-for-booking` `mobile-get-all-doctors` `mobile-get-accessible-doctors` `mobile-get-patient-appointments` `mobile-get-patient-doctors` `mobile-get-doctor-all-appointments`

`mobile-get-doctor-today-appointments` `mobile-handle-appointment-action`
`mobile-update-appointment-status` `mobile-update-appointment-notification` `mobile-get-doctor-clinics`

Patient Records & Billing

`mobile-get-patient-profile` `mobile-update-patient-profile` `mobile-update-profile`
`mobile-upload-profile-image` `mobile-load-user-profile-image` `mobile-get-patient-prescriptions`
`mobile-get-patient-invoices` `mobile-get-invoice-detail` `mobile-get-patient-statement`
`mobile-get-patient-packages` `mobile-get-package-detail` `mobile-get-orders`
`mobile-get-patient-eye-tests` `mobile-get-patient-eye-summary` `mobile-get-dental-encounters`
`mobile-get-nutrition-overview` `mobile-scan-card`

Doctor Dashboard & Records

`mobile-get-doctor-dashboard-stats` `mobile-get-dashboard-stats` `mobile-get-doctor-patients`
`mobile-get-doctor-prescriptions` `mobile-get-doctor-orders` `mobile-get-doctor-dental-encounters`
`mobile-get-doctor-nutrition-plans` `mobile-get-doctor-vision-tests` `mobile-doctor-manage-vision-test`
`mobile-get-finance-summary`

Questionnaires & Q&A

`mobile-add-question` `mobile-update-question` `mobile-delete-question` `mobile-reorder-questions`
`mobile-create-questionnaire` `mobile-update-questionnaire` `mobile-delete-questionnaire`
`mobile-get-questionnaires` `mobile-get-doctor-questionnaires` `mobile-assign-questionnaire`
`mobile-unassign-questionnaire` `mobile-get-questionnaire-patients` `mobile-submit-questionnaire`
`mobile-submit-question-response` `mobile-get-patient-answers` `mobile-get-patient-specific-answers`
`mobile-get-doctor-responses`

Scheduling (Time Management)

`mobile-upsert-recurring-schedule` `mobile-delete-recurring-schedule` `mobile-create-schedule-block`
`mobile-update-schedule-block` `mobile-delete-schedule-block` `mobile-create-schedule-exception`
`mobile-update-schedule-exception` `mobile-delete-schedule-exception` `mobile-get-doctor-time-management`

Notifications & Devices

`mobile-get-notifications` `mobile-get-doctor-notifications` `mobile-mark-notification-read`
`mobile-mark-all-notifications-read` `mobile-mark-doctor-notification-read`
`mobile-mark-all-doctor-notifications-read` `mobile-complete-notification`

`mobile-insert-notifications-read` `mobile-toggle-notifications` `mobile-toggle-doctor-notifications` `mobile-save-device-token` `mobile-save-doctor-device-token` `mobile-get-device-token-status` `send-push-notification` `send-doctor-push-notification`

Settings & Help

`mobile-get-biometric-settings` `mobile-update-biometric-settings` `mobile-get-primary-settings` `mobile-get-doctor-settings` `mobile-update-doctor-security-settings` `mobile-update-darkmode` `mobile-get-payment-methods` `mobile-manage-payment-method` `mobile-get-help-content`

---

10. Database Schema

PostgreSQL on Supabase with **Row-Level Security (RLS)** on every app table. The app creates the tables below; it also references existing clinic-system tables (`patients` , `user_patients` , `doctors` , `clinics` , `companies` , `appointments` , `users`).

user_patient_sessions

Custom auth sessions for mobile patients (token validated by edge functions).

Column	Type	Notes
id	uuid PK	gen_random_uuid()
patient_id	uuid FK → user_patients	ON DELETE CASCADE
token	text UNIQUE	session token
expires_at	timestamptz	expiry
created_at / updated_at	timestamptz	defaults now()

otp_verifications

Column	Type	Notes
id	uuid PK	
phone / email	text	recipient
otp_code	text	code
is_verified	boolean	default false
expires_at	timestamptz	+ registration fields added later

device_tokens

FCM push tokens per device (patient & doctor; `patient_id` / `user_id` nullable).

Column	Type	Notes
id	uuid PK	
patient_id / user_id	uuid	owner (nullable)
medical_id	text	quick lookup
fcm_token	text	unique device token
platform	text	ios / android (+web later)
device_model / app_version	text	metadata
is_active	boolean	default true

doctor_schedules · schedule_exceptions · schedule_blocks

Table	Key columns
doctor_schedules	doctor_id, clinic_id, day_of_week (0–6), start_time, end_time, slot_duration, is_active · UNIQUE(doctor,clinic,day)
schedule_exceptions	doctor_id, clinic_id, exception_date, is_available, start_time, end_time, reason · UNIQUE(doctor,clinic,date)
schedule_blocks	doctor_id, clinic_id, blocked time range (lunch, meetings) — recurring or one-time

patient_questions

Column	Type	Notes
id	uuid PK	
patient_id / doctor_id / company_id	uuid FK	relations
question	text	patient question
response	text	doctor answer (nullable)
status	text	pending / answered
created_at / answered_at	timestamptz	

patient_notifications

Column	Type	Notes
id	uuid PK	
company_id / patient_id / objective_id	uuid	relations
medical_id	text	lookup
category	text	appointment / order / prescription ...
message_header / message_body	text	content
read	boolean	default false
usertype / auth_status	text	added later

doctor_notifications

Column	Type	Notes
id	uuid PK	
company_id / doctor_id / patient_id / objective_id	integer	relations (FK to companies/doctors)
category / message	text	content
read	boolean	default false
status	text	pending / completed
completed	text	yes / no / denied (authorization)
auth_status	text	authorization workflow

notification_reads

Tracks per-user read state for appointment notifications. Columns: `user_id`, `appointment_id`, `notification_type` (booked/confirmed/cancelled), `read_at` (null = unread). UNIQUE(user, appointment, type).

patient_payment_methods

Column	Type	Notes
id	uuid PK	
patient_id	uuid FK → user_patients	
brand	text	Visa / Mastercard / Amex / Other
last4	text	masked card
holder_name / expiry	text	MM/YY
is_default	boolean	

> **Profile flags added to existing tables:** `user_patients` and `users` gained `darkmode`, `allow_notifications`, and biometric auth fields; `doctors` gained `allow_whatsapp_chat`. A `patient-profiles` Storage bucket holds profile images.

11. DB Functions & Triggers

RPC functions (dashboard & data)

Role-aware aggregates power the doctor dashboard (admin / doctor / staff variants):

`get_doctor_today_schedule` `get_doctor_pending_today` `get_doctor_completed_today`
`get_doctor_cancelled_count` `get_doctor_scheduled_count` `get_doctor_total_patients`
`get_doctor_patients` `get_doctor_notifications_via_access` `get_admin_* / get_staff_*`
`upsert_device_token`

Notification triggers

Postgres triggers create notification rows automatically on clinical events, then a generic FCM trigger delivers them:

Trigger function	Fires on
notify_patient_new_appointment / notify_appointment_created / notify_appointment_update	appointment changes
notify_new_order / notify_patient_new_order	new order
notify_prescription_created	new prescription
notify_invoice_created	new invoice
notify_dental_encounter_created	dental encounter
notify_eye_test_created / notify_eyeglass_prescription_created	vision records
notify_nutrition_plan_assigned	nutrition plan
notify_questionnaire_assignment	questionnaire assigned
notify_doctor_response	doctor answers a question
auto_send_fcm_notification · send_fcm_on_patient_notification · send_fcm_on_doctor_notification	AFTER INSERT on notifications → pg_net → edge function

> The `pg_net` extension lets Postgres make outbound HTTP calls so a database insert can directly invoke `send-push-notification` — no external cron needed.

12. Push Notifications

1. **Registration:** on login the app gets an FCM token (Firebase) and stores it via `mobile-save-device-token` / `mobile-save-doctor-device-token` into `device_tokens` (secured locally with expo-secure-store).
2. **Generation:** a clinical event inserts a row in `patient_notifications` / `doctor_notifications` (via triggers).
3. **Delivery:** an AFTER-INSERT FCM trigger uses `pg_net` to call the `send-push-notification` edge function, which looks up active device tokens and pushes through FCM.
4. **Handling:** `utils/notifications.ts` wires foreground listeners + a background handler and routes the user to the right screen (deep-link navigation by category).
5. **Sign-out:** the current device token is deactivated (`mobile-logout-device`).

13. Security

- **Row-Level Security** on every app table — patients/doctors only access their own rows; a service-role policy lets edge functions operate.
- **Custom session tokens** (`user_patient_sessions`) sent via `X-Session-Token` ; the anon key never grants direct table access.
- **Password hashing** with @noble/hashes + expo-crypto (`utils/passwordHash.ts`).
- **OTP verification** for registration & password reset (phone/email).
- **Biometric gate** (Face ID / fingerprint) optionally protects app entry.
- **Image CAPTCHA** on registration to block bots.
- **Single / multi-device login** control + token deactivation on logout.
- **Secrets** stored with expo-secure-store; service account keys kept server-side.

> **Note:** `app.json` ships the Supabase URL + *anon* key (safe to expose by design — protected by RLS). Service-role keys and the Firebase service account must remain server-side only and never be committed to a public repo.

14. Build & Deployment

NPM SCRIPTS

Script	Action
npm run dev	Start Expo dev server
npm run android / ios	Native run on device/emulator
npm run build:web	Export web bundle
npm run typecheck	`tsc --noEmit`
npm run lint	Expo lint
qa:frontend / backend / database / mobile	QA agent review scripts

TARGETS & CONFIG

- **App identity:** `com.maw3edak.app` (iOS & Android), scheme `maw3edak`.
- **EAS Build** (`eas.json`) for native binaries; **expo-updates** for OTA updates.
- **Firebase** configured via `google-services.json` (Android) and `googleservice-info.plist` (iOS).
- **New Architecture** enabled; typed routes on; portrait orientation.
- **Permissions:** Camera, Storage, Microphone, remote-notification background mode.

---

15. Key Code Samples

Design tokens — `constants/theme.ts`

```
export const colors = {
  primary: '#006BFF', primaryLight: '#EFF6FF', primaryDark: '#1E40AF',
  success: '#10B981', warning: '#F59E0B', error: '#EF4444', purple: '#8B5CF6',
  background: '#FFFFFF', backgroundSecondary: '#F8FAFC',
  text: { primary: '#1E293B', secondary: '#64748B', tertiary: '#94A3B8' },
};
export const spacing      = { xs:4, sm:8, md:12, lg:16, xl:20, xxl:24, xxxl:32 };
export const borderRadius = { sm:8, md:12, lg:16, xl:20, xxl:24, full:9999 };
export const typography = {
  h1: { fontSize:28, fontFamily:'Inter-Bold',     lineHeight:36 },
  body:{ fontSize:15, fontFamily:'Inter-Regular',  lineHeight:22 },
};
```

Supabase client — `utils/supabase.ts`

```
import { createClient } from '@supabase/supabase-js';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { config } from './config';

export const supabase = createClient(
  config.supabaseUrl,
  config.supabaseAnonKey,
  { auth: {
      storage: AsyncStorage,
      autoRefreshToken: true,
      persistSession: true,
      detectSessionInUrl: false,
  }}
);
```

Authenticated edge-function call — switch clinic (AuthContext)

```
const response = await fetch(
  `${config.supabaseUrl}/functions/v1/mobile-switch-company`,
  { method: 'POST',
    headers: {
      Authorization: `Bearer ${config.supabaseAnonKey}`,
      'X-Session-Token': session.access_token,   // custom session token
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ companyId: clinicId }),
  });
const result = await response.json();   // { success, settings, ... }
```

Root provider tree — `app/_layout.tsx`

```
<ErrorBoundary>
  <SafeAreaProvider>
    <AuthProvider>
      <ThemeProvider>
        <BiometricAuthGate>
          <Stack screenOptions={{ headerShown: false }}>
            <Stack.Screen name="index"  />
            <Stack.Screen name="login"  />
            <Stack.Screen name="(tabs)" />          {/* patient */}
            <Stack.Screen name="(doctor-tabs)" />   {/* doctor  */}
          </Stack>
        </BiometricAuthGate>
      </ThemeProvider>
    </AuthProvider>
  </SafeAreaProvider>
</ErrorBoundary>
```

Patient tab bar — `app/(tabs)/_layout.tsx`

```
<Tabs screenOptions={{ tabBarActiveTintColor: colors.primary }}>
  <Tabs.Screen name="index"         options={{ title:'Home',    icon:Home }} />
  <Tabs.Screen name="appointments"  options={{ title:'Appts',   icon:Calendar }} />
  <Tabs.Screen name="visit-history" options={{ title:'Visits',  icon:History }} />
  {/* profile, orders, invoices, ... → href:null (hidden from tab bar) */}
</Tabs>
```

Auto-deliver FCM on new notification (DB trigger pattern)

```
CREATE OR REPLACE FUNCTION send_fcm_on_patient_notification()
RETURNS TRIGGER AS $$
BEGIN
  PERFORM net.http_post(
    url     := 'https://<project>.supabase.co/functions/v1/send-push-notification',
    headers := '{"Content-Type":"application/json"}'::jsonb,
    body    := json_build_object(
        'patient_id', NEW.patient_id,
        'title',      NEW.message_header,
        'body',       NEW.message_body,
        'category',   NEW.category)::jsonb
  );
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_fcm_patient_notification
  AFTER INSERT ON patient_notifications
  FOR EACH ROW EXECUTE FUNCTION send_fcm_on_patient_notification();
```

To export: open this file in a browser and choose **Print** → **Save as PDF** (or the 🖨 button). Set margins to "Default" and enable "Background graphics".